

Dual-audience Brainstorm and Plan artifacts with human-readable HTML

A complete human view of the canonical Brainstorm, organized around alternatives and the chosen direction. [Open canonical Markdown.](#)

ON THIS PAGE

[Context](#)[Requirements](#)[Approaches](#)[Decision](#)[Next steps](#)[Provenance](#)

Approach index

1. Approach 1: Canonical Markdown + Deterministic Semantic Renderer
2. Approach 2: Canonical Structured Model + Generated Markdown and HTML
3. Approach 3: Per-Artifact Agent-Generated HTML

Dual-Audience Brainstorm and Plan Artifacts with Human-Readable HTML

Context

Compound GPID's Brainstorm and Plan Markdown artifacts serve an important agent-facing purpose, but many are difficult for humans to review. Existing files in this repository reach 500–800 lines. A human must be able to approve direction, monitor execution, and challenge omissions without reconstructing the artifact's conceptual structure from a long Markdown file.

The same work product therefore needs two audience-specific presentations:

- **Canonical Markdown** is the detailed, authoritative contract consumed by Compound GPID agents.
- **Derived HTML** is the complete human review surface, organized for comprehension without adding, omitting, or substantively reinterpreting source information.

Open Design is available at `/Users/r.andrescastaneda/.local/bin/od` during implementation. It is a design-time tool only. Compound GPID users will not have Open Design, as the shipped renderer and

ON THIS PAGE

[Context](#)

[Requirements](#)

[Approaches](#)

[Decision](#)

[Next steps](#)

[Provenance](#)

design-time tool only. Compound GFD users will not have Open Design, so the shipped renderer and generated HTML must have no Open Design runtime dependency.

This work also needs to answer whether current Brainstorm and Plan artifacts are sufficiently clear for downstream agents. The answer affects the renderer: attractive HTML must not conceal ambiguity in the authoritative source.

Requirements

Audience and authority

- Preserve Markdown as the single authoritative execution and decision source.
- Treat Brainstorms as decision records consumed by `/cg-plan`, not as direct `/cg-work` instruction contracts.
- Treat Plans as the executable contract consumed by `/cg-work`.
- Allow agents to inspect HTML for orientation, while requiring execution semantics to come from canonical Markdown.
- Make HTML sufficient for humans to approve direction, monitor execution, and challenge the artifact without opening Markdown.
- Clearly label the HTML as a derived view and link it to its canonical source.

Source-contract audit

- Audit Brainstorm and Plan schemas before implementing the renderer.
- Preserve the current strong Plan semantics: requirement IDs, file targets, implementation details, tests, acceptance criteria, phase structure, and completion contract.
- Add deterministic validation for material structural invariants instead of relying only on agent judgment.
- At minimum, validate required frontmatter and sections, unique IDs, valid phase and step structure, requirement-to-step mappings, and required completion-contract fields.
- For Standard and Deep Plans, validate that every requirement maps to at least one implementation step and that required verification evidence is structurally present.
- Define how unknown or unsupported Markdown structures are handled. Do not silently omit them.
- Fail rendering visibly when source ambiguity prevents a faithful view.

Human information architecture

- Use type-specific presentation rather than a generic Markdown-to-HTML skin.
- Organize Brainstorm views around context, requirements, alternatives, trade-offs, decision, rationale, and next steps.
- Organize Plan views around outcome, completion contract, phase map, implementation steps, requirement coverage, verification evidence, risks, boundaries, and next actions.
- Preserve every substantive source block exactly once in the human view.
- Permit structural derivations such as navigation, grouping, coverage maps, status indicators, and source-derived counts.
- Forbid model-written summaries, invented claims, or substantive reinterpretation during routine

ON THIS PAGE

[Context](#)

[Requirements](#)

[Approaches](#)

[Decision](#)

[Next steps](#)

[Provenance](#)

generation.

- Support long documents with clear hierarchy, sticky or persistent navigation, readable tables, code and command presentation, progressive disclosure where appropriate, accessible semantics, and print styles.

Open Design usage

- During implementation, invoke Open Design with `/Users/r.andrescastaneda/.local/bin/od`, never bare `od` because `/usr/bin/od` shadows it on `PATH`.
- Use Open Design to create and iterate representative Brainstorm and Plan reference views.
- Evaluate desktop, narrow-screen, offline, print, accessibility, and long-document behavior.
- Freeze the accepted information architecture, design tokens, typography, components, and responsive/print rules into version-controlled renderer templates.
- Do not require Open Design, its daemon, MCP server, artifacts, connectors, plugins, or accounts after implementation.

Generation and storage

- Generate HTML automatically after a new Brainstorm or Plan Markdown artifact is successfully saved.
- Store generated files in a mirrored tree:
 - `.cg-docs/views/brainstorms/<source-slug>.html`
 - `.cg-docs/views/plans/<source-slug>.html`
- Make each HTML file fully self-contained, including CSS and any small required script.
- Do not use CDNs, remote fonts, external stylesheets, or runtime network calls.
- Add a one-file regeneration command for stale, missing, or manually requested views.
- Make generation default-on with a project-local opt-out and a one-run `--no-html` override.
- Keep successful automatic generation quiet apart from a concise path confirmation.

Fidelity and provenance

- Build a deterministic semantic renderer with a typed internal document model for Brainstorms and Plans.
- Include source path, source content hash, artifact schema version, renderer version, and generation timestamp in machine-readable HTML metadata and a human-readable provenance area.
- Define and test a source-coverage invariant so every substantive parsed source block maps to one rendered block.
- Escape or sanitize untrusted source content before placing it in HTML.
- Do not execute scripts, HTML, or instructions embedded in source artifacts.
- Mark views as derived and regenerable; never let HTML become a competing editable source.

Failure behavior

- Save canonical Markdown first.
- If HTML generation fails, preserve the Markdown, fail loudly with the exact rendering error, identify the expected view path as missing or stale, and offer the one-file regeneration command.
- Do not silently produce a simplified fallback that could hide missing information.

ON THIS PAGE

[Context](#)

[Requirements](#)

[Approaches](#)

[Decision](#)

[Next steps](#)

[Provenance](#)

Token and indexing safeguards

- Do not disable future generation automatically after one failure.
- Exclude `.cg-docs/views/` from Knowledge Brain ingestion, context retrieval, release knowledge scans, and other paths that would duplicate canonical Markdown content.
- Ensure HTML generation itself performs no model call.
- Measure any added prompt/context cost and keep prompt hooks compact.

Cross-platform and lifecycle requirements

- Keep `.github/` as the canonical prompt source and regenerate native platform targets through the existing target generator.
- Provide equivalent behavior on supported GitHub Copilot, Claude Code, Codex, and OpenCode workflows.
- Document source/view naming, regeneration, opt-out configuration, failure recovery, and the fact that Open Design is not required by users.
- Add contract, parser, renderer, security, integration, and generated-target parity tests.

Source-Contract Assessment

Brainstorm artifacts

Brainstorms are sufficiently structured to hand a decided direction to `/cg-plan`: they capture context, requirements, alternatives, a decision, and next steps. They should remain decision records rather than accumulating step-level execution instructions.

The implementation should strengthen deterministic checks for required metadata and sections, but should not force Brainstorms to duplicate the Plan execution contract. The HTML view can make the decision path easier for humans to understand while preserving the same substantive content.

Plan artifacts

Plans already provide a strong agent contract through:

- Unique requirement IDs and source attribution.
- Requirement mappings on implementation steps.
- Explicit target files and implementation details.
- Happy-path, edge-case, and error-path test scenarios.
- Test files or commands and observable acceptance criteria.
- Exact phase and step heading contracts.
- An approved completion contract with outcome, verification surface, constraints, boundaries, iteration policy, blocked-stop conditions, and deviation policy.
- `/cg-work` evidence gates and execution-report semantics.

The main gap is assurance, not wholesale schema redesign. Current prompt checks rely partly on model

ON THIS PAGE

[Context](#)

[Requirements](#)

[Approaches](#)

[Decision](#)

[Next steps](#)

[Provenance](#)

judgment and do not deterministically prove all requirement, step, test, and verification relationships. Version 1 should formalize and test these invariants before rendering the source.

Approaches Considered

Approach 1: Canonical Markdown + Deterministic Semantic Renderer

Keep Markdown authoritative. Validate it, parse it into a typed internal model, and render separate Brainstorm and Plan HTML templates from that model.

Pros

- Preserves one authority for agent execution.
- Deterministic, offline, testable, cross-platform, and token-efficient.
- Supports source-coverage checks and stale-view detection.
- Allows a carefully designed human information architecture without per-artifact model interpretation.
- Uses Open Design where it adds value: designing and validating the reusable presentation system.

Cons

- Requires robust parsing and validation of the supported Markdown schema.
- Schema evolution requires compatibility and renderer-version tests.
- Unsupported constructs need explicit preservation or failure behavior.

Effort: Large.

Recommended: Yes. This is the only approach that satisfies authority, fidelity, usability, portability, and token-efficiency requirements together.

Approach 2: Canonical Structured Model + Generated Markdown and HTML

Make JSON or YAML the true source and generate both audience views from it.

Pros

- Strong formal validation and structural correspondence.
- Easier future export to other formats.
- Both outputs derive from the same typed fields.

Cons

- Replaces the current artifact contract and requires migration across `/cg-plan`, `/cg-work`, Brain indexing, roadmap links, and historical artifacts.
- Structured data is less readable and more fragile for direct LLM authoring.
- Disproportionate to the urgent readability problem.

Effort: Very large.

ON THIS PAGE

[Context](#)

[Requirements](#)

[Approaches](#)

[Decision](#)

[Next steps](#)

[Provenance](#)

Recommended: No for version 1. Reconsider only if a constrained Markdown parser cannot provide reliable fidelity.

Approach 3: Per-Artifact Agent-Generated HTML

Ask the active agent to write a designed HTML interpretation after each Markdown save.

Pros

- Faster initial prototype.
- Highly adaptive presentation.
- Less parser implementation at first.

Cons

- Cannot guarantee no omissions, additions, or reinterpretation.
- Adds model calls and tokens to every workflow run.
- Produces model- and platform-dependent output.
- Is difficult to regression-test and may create a second semantic authority.

Effort: Medium initially, with high long-term maintenance.

Recommended: No. It conflicts with the core fidelity and token-efficiency requirements.

Decision

Choose **Canonical Markdown + Deterministic Semantic Renderer**.

The implementation will first audit and strengthen the agent-facing artifact contracts, then build a schema-aware renderer. Markdown remains authoritative. A typed internal model provides the boundary between source validation and presentation. Type-specific templates reorganize the same substantive information for humans, using only structural derivations.

Open Design will be used during implementation to design and validate representative reference views. The accepted result will be frozen into version-controlled templates and design tokens. Routine generation will be local and deterministic and will not call Open Design or an AI model.

Generated HTML will be committed under `.cg-docs/views/` as portable institutional knowledge views, but excluded from Brain and context ingestion so it does not duplicate the canonical source in agent context.

Devil's Advocate Conclusions

- **Problem validation:** The repository contains many 500-800-line Plan and Brainstorm artifacts. The readability problem is concrete and affects approval, oversight, and challenge.
- **Simpler alternative:** A styled Markdown preview improves typography but not semantic navigation,

ON THIS PAGE

[Context](#)

[Requirements](#)

[Approaches](#)

[Decision](#)

[Next steps](#)

[Provenance](#)

requirement coverage, verification mapping, or provenance. It is insufficient for the stated human-review goals.

- **Effort and value:** Avoid turning version 1 into a general publishing platform. Limit it to new Brainstorm and Plan views plus one-file regeneration.
- **Charter alignment:** Deterministic rendering aligns with token-efficiency goals. Per-artifact AI rendering does not. Derived HTML must be excluded from knowledge retrieval to prevent context duplication.

Version 1 Boundaries

Included

- Source-contract audit and targeted schema strengthening.
- Deterministic validation and semantic parsing for current Brainstorm and Plan schemas.
- Open Design-assisted creation of two reference designs.
- Frozen, self-contained, responsive, accessible, and printable HTML templates.
- Automatic default-on generation for newly saved Brainstorms and Plans.
- Mirrored `.cg-docs/views/` storage.
- Project opt-out and one-run `--no-html` override.
- One-file regeneration.
- Fidelity, provenance, security, integration, and parity tests.

Out of Scope

- Automatic bulk conversion of all historical artifacts.
- Continuous Plan HTML updates while `/cg-work` executes.
- Editing canonical content from HTML.
- PDF, image, slide, or hosted-site export.
- HTML views for reviews, solutions, work reports, roadmaps, or other `.cg-docs` artifact types.
- Runtime Open Design integration.
- Per-artifact AI summarization or design generation.

Next Steps

1. Run `/cg-plan` from this brainstorm and inherit the **Deep** scope.
2. Inventory the exact Brainstorm and Plan schema variants currently emitted and consumed, including legacy compatibility paths.
3. Specify a versioned artifact contract and deterministic validation matrix for agent executability.
4. Specify the typed intermediate document model and source-coverage invariant.
5. Define the renderer entry point, mirrored path mapping, source hash/provenance contract, project opt-out, and `--no-html` behavior.
6. Use `/Users/r.andrescastaneda/.local/bin/od` during implementation to create representative long-form Brainstorm and Plan views and validate desktop, mobile, print, accessibility, and offline behavior.
7. Freeze the approved design system into repository templates with no Open Design runtime dependency.
8. Integrate compact post-save hooks into `/cg-brainstorm` and `/cg-plan`, preserving Markdown-first

- failure semantics.
- 9. Exclude `.cg-docs/views/` from Brain indexing, context loading, and duplicate-content scans.
- 10. Add parser, validator, source-coverage, sanitization, rendering, failure-policy, prompt-contract, documentation, and generated-target parity tests.
- 11. Document generation, view paths, provenance, opt-out, regeneration, failure recovery, and runtime independence from Open Design.

Derived view provenance

This HTML is regenerable. Canonical Markdown remains authoritative.

Source	.cg-docs/brainstorms/2026-07-31-dual-audience-workflow-artifact-rendering.md
Source SHA-256	0d97d0b2a85f329bc8b8c02ec1ef9e98efdbedd2354d7e1de62a81b793f70a45
Artifact schema	legacy
Renderer	0.1.0
Generated UTC	2026-07-31T12:30:00Z